Introduzione ai DB No SQL

Python

Un Database, o base di dati, è un insieme organizzato di dati strutturati e correlati tra loro, progettato per essere facilmente accessibile, gestito e aggiornato.

I database vengono utilizzati per memorizzare informazioni in modo persistente, consentendo operazioni di inserimento, modifica, cancellazione e interrogazione dei dati.

Questi sistemi sono fondamentali in moltissimi ambiti, dal software gestionale ai siti web, passando per applicazioni mobile e sistemi industriali.

L’accesso e la gestione dei database sono solitamente affidati a un DBMS (Database Management System), ovvero un software che si occupa dell'interazione tra l'utente e il database.

Database relazionali (RDBMS)

- Basati su tabelle (righe e colonne), utilizzano il linguaggio SQL. Ogni tabella ha relazioni con altre tramite chiavi. Esempi: MySQL, PostgreSQL, Oracle.

Database non relazionali (NoSQL)

- Progettati per grandi quantità di dati non strutturati o semi-strutturati. Non usano lo schema rigido delle tabelle. Tipologie comuni:
- Documentali: archiviano dati in formato JSON o BSON (es. MongoDB)
- Key-Value: semplici coppie chiave-valore (es. Redis)
- Colonnari: ottimizzati per letture su grandi volumi (es. Cassandra)
- A grafo: rappresentano dati e relazioni con nodi e archi (es. Neo4j)

Schema flessibile

- A differenza dei database relazionali, i NoSQL non richiedono uno schema rigido. I dati possono avere strutture diverse anche all’interno della stessa collezione.

Scalabilità orizzontale

- Possono essere facilmente distribuiti su più server (cluster), permettendo di gestire grandi volumi di dati aumentando il numero di macchine.

Alte prestazioni

- Sono ottimizzati per operazioni rapide di lettura e scrittura, soprattutto in contesti con grandi quantità di dati e traffico elevato.

Modelli di dati diversi

- Supportano vari modelli oltre alle tabelle: documenti (es. JSON), chiave-valore, grafi e colonne larghe, adattandosi a diversi tipi di applicazioni.

I database NoSQL nascono per rispondere ai limiti dei sistemi relazionali tradizionali in contesti moderni come applicazioni web scalabili, big data e sistemi distribuiti.

A differenza dei database SQL, offrono uno schema flessibile, permettono di gestire grandi volumi di dati su più nodi (scalabilità orizzontale) e supportano diversi modelli di rappresentazione (documenti, chiave-valore, grafi, colonne). Questo li rende particolarmente adatti quando i dati sono eterogenei o cambiano frequentemente.

	SQL (Relazionali)	NoSQL
Schema	Rigido, predefinito	Flessibile, dinamico
Struttura dati	Tabelle (righe e colonne)	Documenti, chiave-valore, grafi, colonne
Scalabilità	Verticale (più potente hardware)	Orizzontale (più nodi)
Consistenza	Forte (ACID)	Eventuale (BASE, spesso)
Prestazioni	Ottime per relazioni complesse	Ottime per grandi volumi e accessi rapidi
Linguaggio	SQL standard	Query specifiche per ogni DB

	Struttura dati	Esempi principali	Vantaggi principali	Svantaggi principali	Casi d’uso tipici
Key-Value	Coppie chiave → valore	Redis, Amazon DynamoDB	Estremamente veloce, semplice	Query limitate (solo per chiave)	Cache, sessioni, carrelli e-commerce
Document-oriented	Documenti JSON/BSON	MongoDB, CouchDB	Schema flessibile, facile da usare	Meno efficiente per relazioni complesse	Web app, API REST, gestione contenuti
Column-family	Colonne distribuite	Apache Cassandra, HBase	Alta scalabilità, ottimo per big data	Complesso da progettare	Analisi dati, sistemi distribuiti
Graph database	Nodi e relazioni (grafi)	Neo4j, ArangoDB	Ottimo per relazioni complesse	Scalabilità più difficile rispetto gli altri	Social network, raccomandazioni, reti

Introduzione a MongoDB

MongoDB è uno dei database NoSQL più diffusi e appartiene alla categoria dei database document-oriented.

Invece di usare tabelle e righe, memorizza i dati in documenti in formato JSON (più precisamente BSON), rendendolo molto intuitivo per chi sviluppa applicazioni moderne.

Le principali caratteristiche di MongoDB includono:

- **Schema dinamico: ogni documento può avere campi diversi**
- **Alta scalabilità: supporta facilmente cluster e distribuzione dei dati**
- **Query potenti: permette ricerche complesse pur mantenendo buone prestazioni**
- **Integrazione con linguaggi moderni: molto usato con JavaScript/Node.js**

Per usare MongoDB in Python si utilizza la libreria pymongo:

1. *pip install pymongo*

Connessione di base:

1. *from pymongo import MongoClient*
2.
3. *client = MongoClient("mongodb://localhost:27017/")*
4. *db = client["scuola"]*
5. *collezione = db["utenti"]*

Documenti e collezioni (struttura dati)

In MongoDB i dati sono salvati in documenti (simili a oggetti JSON) dentro collezioni.

```
{
  "nome": "Luca",
  "eta": 25,
  "email": "luca@email.com",
  "hobby": ["gaming", "coding"]
}
```

Non serve definire prima uno schema: un altro documento nella stessa collezione potrebbe avere campi diversi.

Inserimento dati (Create)

```
collezione.insert_one({
    "nome": "Anna",
    "eta": 30,
    "email": "anna@email.com"
})
```

Inserisce un documento nella collezione.

Se non esiste, MongoDB la crea automaticamente.

Query (Read)

```
1. for utente in collezione.find({"eta": {"$gt": 20}}):
2.     print(utente)
```

Restituisce tutti gli utenti con età maggiore di 20 anni.

Query (Read)

```
1.db.utenti.find({ eta: { $gt: 20 } })
```

Recupera tutti gli utenti con età maggiore di 20 anni.
MongoDB usa operatori come $gt, $lt, $eq.

Aggiornamento (Update)

```
collezione.update_one(
    {"nome": "Anna"},
    {"$set": {"eta": 31}}
)
```

Aggiorna il campo eta per il documento selezionato.

Eliminazione (Delete)

```
1.collezione.delete_one({"nome": "Anna"})
```

Rimuove il documento che soddisfa la condizione.

Schema flessibile

```
collezione.insert_one({
    "nome": "Marco",
    "skills": ["Python", "AI"]
})
```

Non serve che tutti i documenti abbiano gli stessi campi.

Indicizzazione (Indexing)

```
1.collezione.create_index("email")
```

Crea un indice per migliorare le prestazioni delle query.

Aggregazione

```
pipeline = [
    {"$group": {"_id": "$eta", "totale": {"$sum": 1}}}
]

for risultato in collezione.aggregate(pipeline):
    print(risultato)
```

Crea un indice per migliorare le prestazioni delle query.

Connessione e lettura singolo document

```
1. utente = collezione.find_one({"nome": "Marco"})
2. print(utente)
```

Restituisce un solo documento che soddisfa la query.

- **Il programma si collega a MongoDB tramite MongoClient, poi seleziona il database scuola e la collezione studenti.**
- **Con insert_many() vengono inseriti più documenti, cioè più studenti. Ogni studente è rappresentato come un dizionario Python, simile a un oggetto JSON.**
- **Con find() si cercano tutti gli studenti appartenenti alla classe 4A.**
- **Con update_one() si modifica un solo documento. Nel primo caso viene cambiata l'età di Anna, mentre nel secondo viene aggiunto un voto alla lista di Marco usando $push.**
- **Con aggregate() si esegue un'operazione più avanzata: gli studenti vengono raggruppati per classe e viene contato quanti studenti ci sono in ogni classe.**
- **Infine, con delete_one() viene eliminato lo studente Luca dalla collezione.**

```python
from pymongo import MongoClient

# 1. Connessione al server MongoDB
client = MongoClient("mongodb://localhost:27017/")

# 2. Creazione / selezione del database
db = client["scuola"]

# 3. Creazione / selezione della collezione
studenti = db["studenti"]

# 4. Inserimento di più studenti
studenti.insert_many([
    {
        "nome": "Anna",
        "eta": 17,
        "classe": "4A",
        "voti": [8, 7, 9]
    },
    {
        "nome": "Marco",
        "eta": 18,
        "classe": "5B",
        "voti": [6, 7, 6]
    },
    {
        "nome": "Luca",
        "eta": 17,
        "classe": "4A",
        "voti": [9, 8, 10]
    }
])

# 5. Lettura degli studenti della classe 4A
print("Studenti della classe 4A:")
for studente in studenti.find({"classe": "4A"}):
    print(studente)

# 6. Aggiornamento dell'età di Anna
studenti.update_one(
    {"nome": "Anna"},
    {"$set": {"eta": 18}}
)

# 7. Aggiunta di un nuovo voto a Marco
studenti.update_one(
    {"nome": "Marco"},
    {"$push": {"voti": 8}}
)

# 8. Calcolo del numero di studenti per classe
pipeline = [
    {
        "$group": {
            "_id": "$classe",
            "totale_studenti": {"$sum": 1}
        }
    }
]

print("Numero studenti per classe:")
for risultato in studenti.aggregate(pipeline):
    print(risultato)

# 9. Eliminazione di uno studente
studenti.delete_one({"nome": "Luca"})
```

Buon MasterD a tutti